Assignment 2: Git & GitHub Fundamentals, Group B5

- **Group Size:** 3
- **Chosen Theme:** Unit Converter
- **Full Names:** Khalil Berjaoui, Rayan Day, Mohammed Taha Mtouri
- **Student IDs:** 124433, 119946, 102913
- **Course Codes & Sections:** CSC 3324 (Group B)
- **GitHub Usernames:** @KatAIml, @rayanday5, @tahamt4
- **GitHub Repository URL:** https://github.com/KatAIml/unit-converter-b5
- **GitHub Pages URL:** https://kataiml.github.io/unit-converter-b5/

1. Project & Role Distribution

We followed the **3-member path** and built a Python Unit Converter with three comparable feature areas: Feature A, length conversion (Khalil); Feature B, temperature conversion (Rayan); Feature C, weight conversion (Taha). Each feature validates its input (non-numbers, unknown units, physical limits) and has its own pytest file. Khalil created the public repository, added Rayan and Taha as collaborators, pushed the initial structure (README, .gitignore, requirements.txt, pytest.ini, workflow, docs page) and later added a CLI menu (`app.py`).

2. Branches, Commits & Contributions

- **Khalil, `feature/length`:** "Add length conversion feature" (`converter/length.py` and README status line), "Add unit tests for length conversion". Later `feature/final-page`: "Add CLI menu, publish v1.0 page and Pages URL".
- **Rayan, `feature/temperature`:** "Add temperature conversion feature", "Add unit tests for temperature conversion", "Fix Celsius to Fahrenheit formula", plus the merge commit "Resolve README project status merge conflict".
- **Taha, `feature/weight`:** "Add weight conversion feature", "Add unit tests for weight conversion", "Add negative weight test requested in review".

3. Pull Requests & Code Review

PR	Author	Branch	Reviewer
#1 Feature A: length conversion	Khalil	`feature/length`	Rayan (line comment, approved)
#2 Feature C: weight conversion	Taha	`feature/weight`	Khalil (Request changes, then approved)
#3 Feature B: temperature conversion	Rayan	`feature/temperature`	Taha (conflict-resolution review, approved)
#4 Final: CLI menu and v1.0 project page	Khalil	`feature/final-page`	Rayan (approved)

Request changes cycle (PR #2): in *Files changed*, Khalil commented on the line `raise ValueError("weight cannot be negative")` that this rule had no test, and submitted a **Request changes** review, which blocked the merge. Taha added `test_negative_weight_raises` on the **same** `feature/weight` branch, pushed it to the existing PR and asked for a re-review. Khalil approved ("Change addressed, approved") and the PR was merged once the `test` check was green.

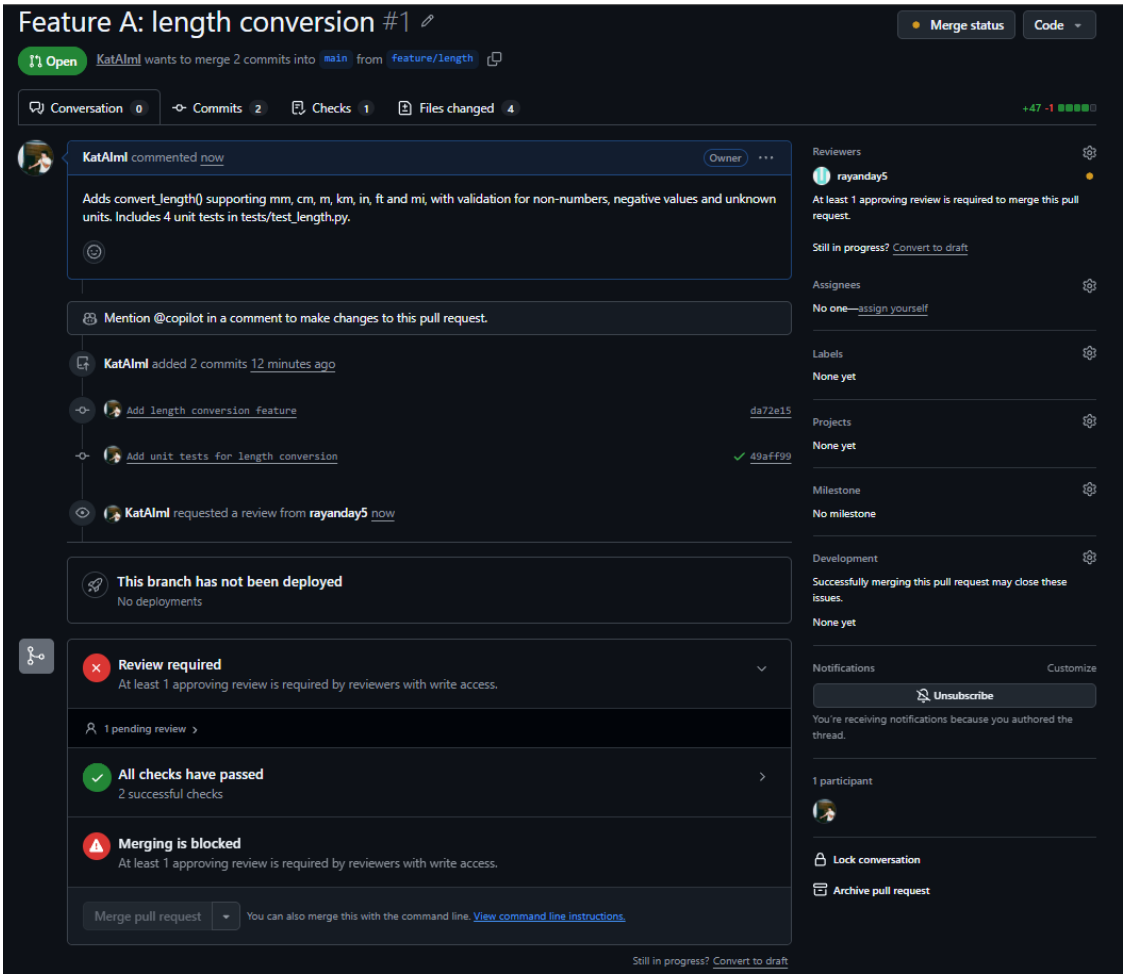


Figure 1. PR #1 opened by Khalil: checks passed, merging blocked until an approving review.

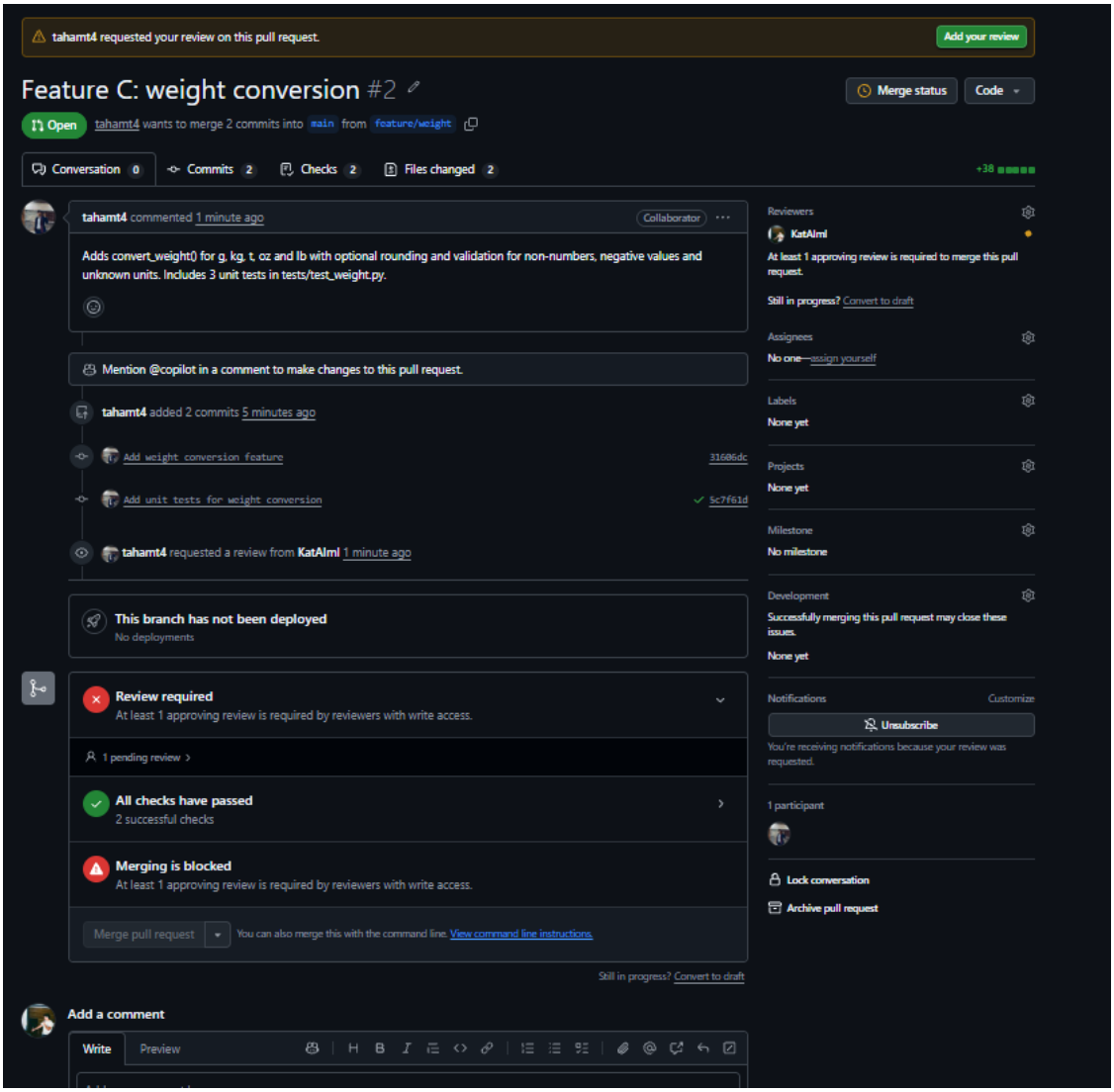


Figure 2. PR #2 opened by Taha: review required, merging blocked.

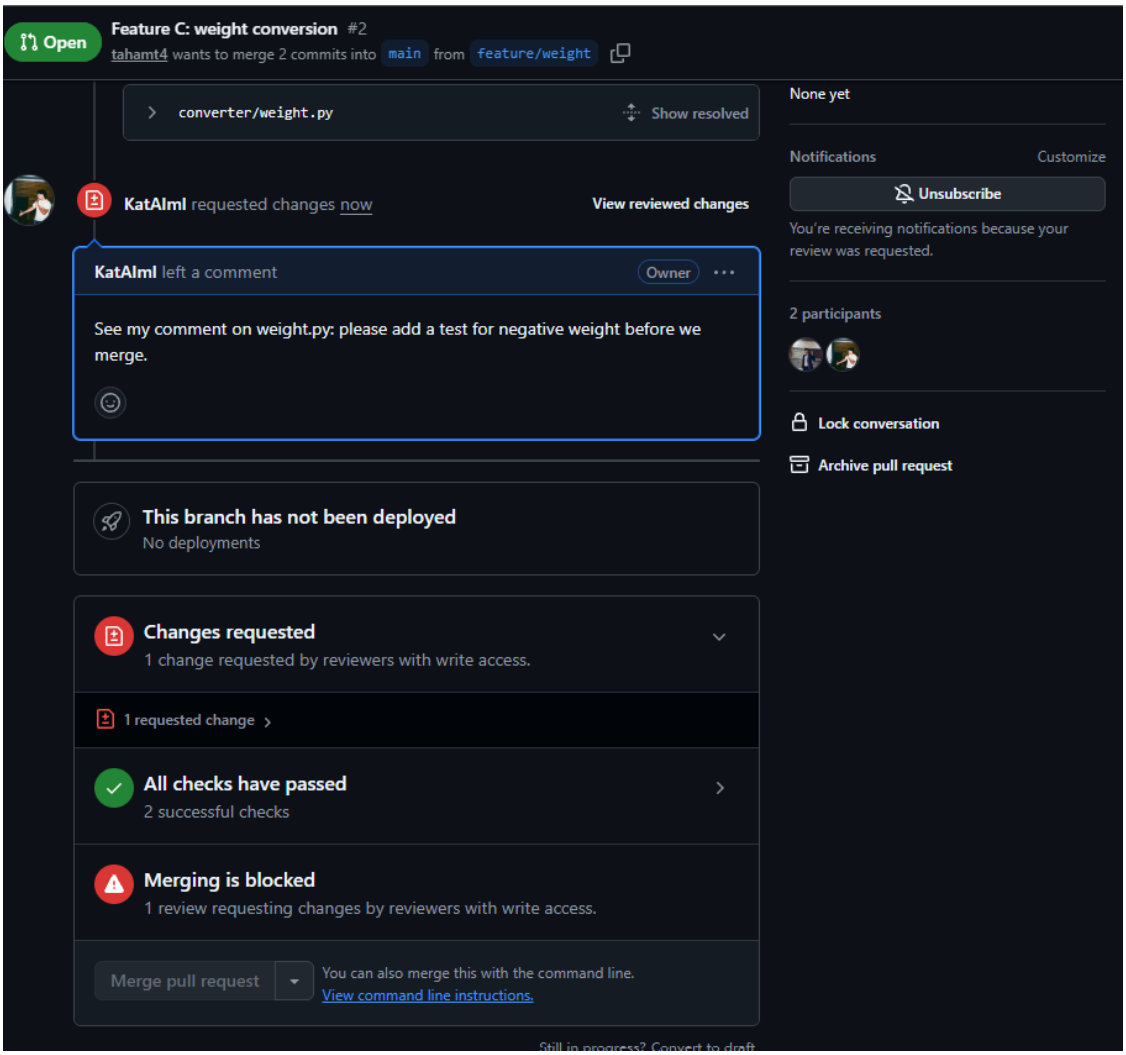


Figure 3. PR #2: Khalil requests changes, merge blocked.

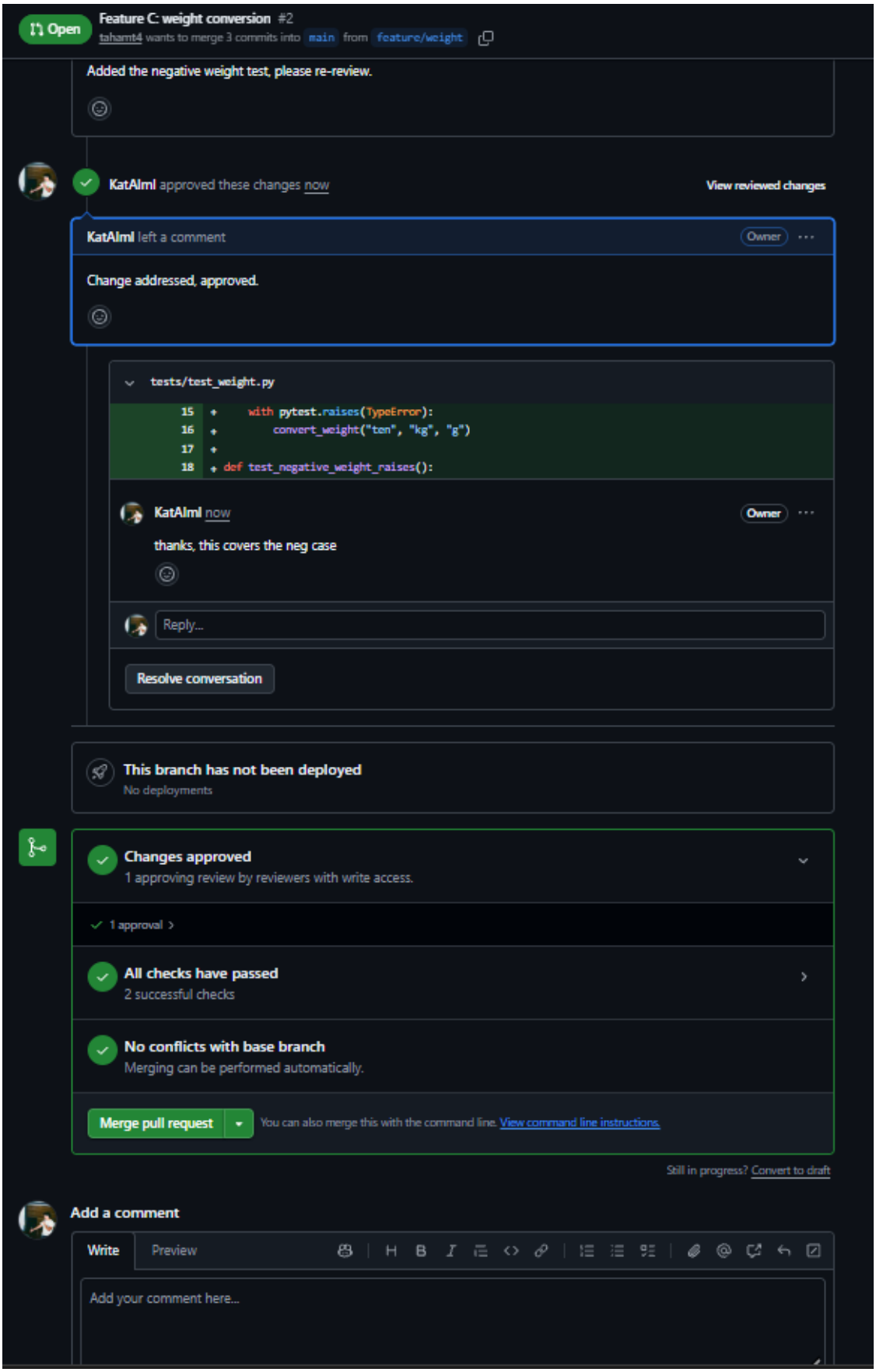

Open
Feature C: weight conversion #2
tahamt4 wants to merge 3 commits into main from feature/weight
Added the negative weight test, please re-review.
KatAIml approved these changes now
View reviewed changes
KatAIml left a comment
Owner
Change addressed, approved.
tests/test_weight.py
15 + with pytest.raises(TypeError):
16 + convert_weight("ton", "kg", "g")
17 +
18 + def test_negative_weight_raises():
KatAIml now
Owner
thanks, this covers the neg case
Reply...
Resolve conversation
This branch has not been deployed
No deployments
Changes approved
1 approving review by reviewers with write access.
1 approval
All checks have passed
2 successful checks
No conflicts with base branch
Merging can be performed automatically.
Merge pull request
You can also merge this with the command line. View command line instructions.
Still in progress? Convert to draft
Add a comment
Write
Preview
Add your comment here...

Figure 4. PR #2 after the fix on the same branch: changes approved, checks passed.

4. Merge Conflict

`main` contained the line `Project status: Development in progress.` Khalil (on `feature/length`) and Rayan (on `feature/temperature`) both changed this exact line. Khalil's PR #1 was merged first, changing it to "Length feature ready for review." Rayan then ran `git fetch origin` and `git merge origin/main`, and Git reported `CONFLICT (content): Merge conflict in README.md`. **Current** (HEAD) was Rayan's line, "Project status: Temperature feature ready for review." **Incoming** (origin/main) was Khalil's line, "Project status: Length feature ready for review." Since by then all three features were in place, we rewrote both into one line: `Project status: Length, weight and temperature features completed.` Rayan removed the markers, staged the file, committed "Resolve README project status merge conflict" and pushed. Taha reviewed and approved PR #3.

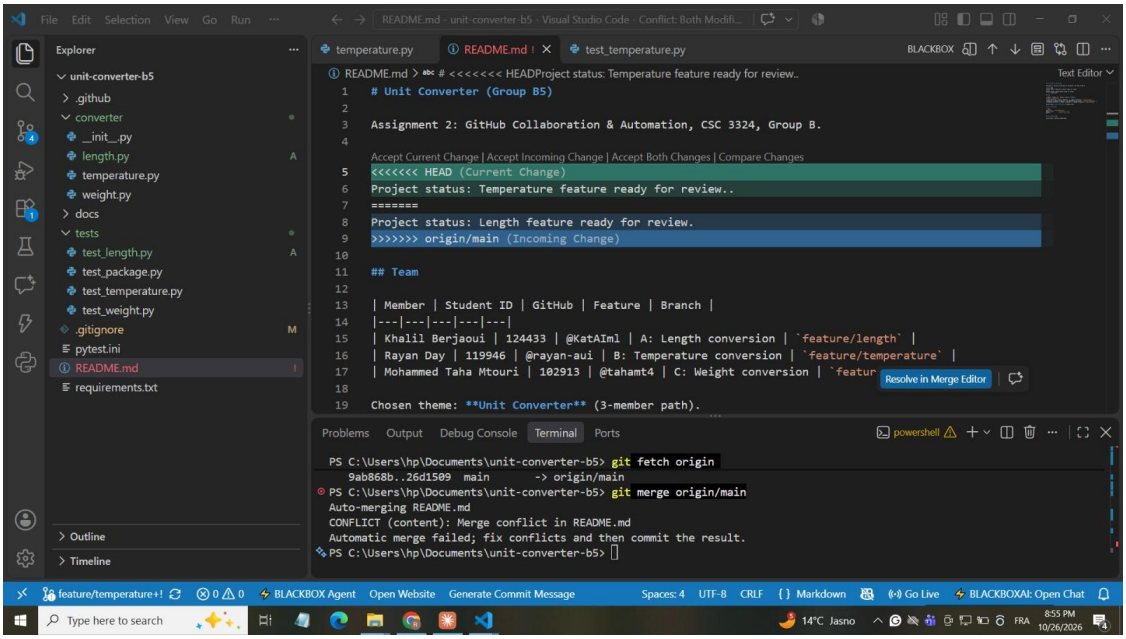


Figure 5. Conflict in README.md after merging origin/main into feature/temperature (Current vs Incoming).

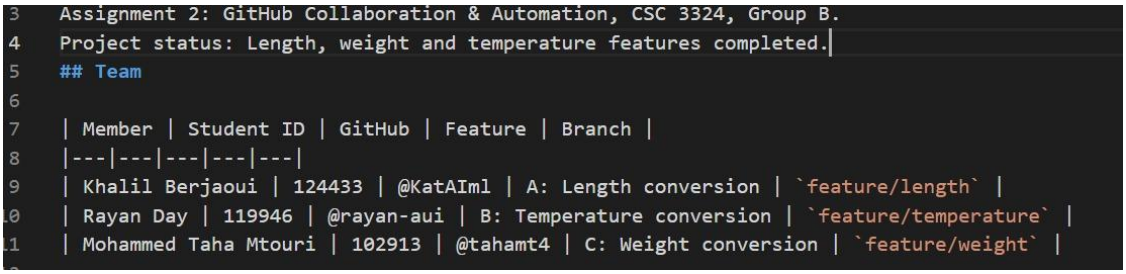


Figure 6. Resolved status line before committing the resolution.

5. Unit Tests

The final suite has **13 pytest tests** in `tests/`: 4 for length, 4 for temperature, 4 for weight (including the one added after review) and 1 package smoke test. All 13 pass locally (`python -m pytest -v`) and in GitHub Actions.

- **Normal behaviour:** 2 km gives 2000 m; 100 °C gives 212 °F; 32 °F gives 0 °C; 1 kg gives 2.2 lb (rounded).
- **Edge/boundary:** 0 K converts to -273.15 °C and is allowed; zero length and weight; same unit to same unit.
- **Invalid input/error:** unknown unit raises `ValueError`; below absolute zero raises `ValueError`; negative length/weight raise `ValueError`; text input raises `TypeError`.

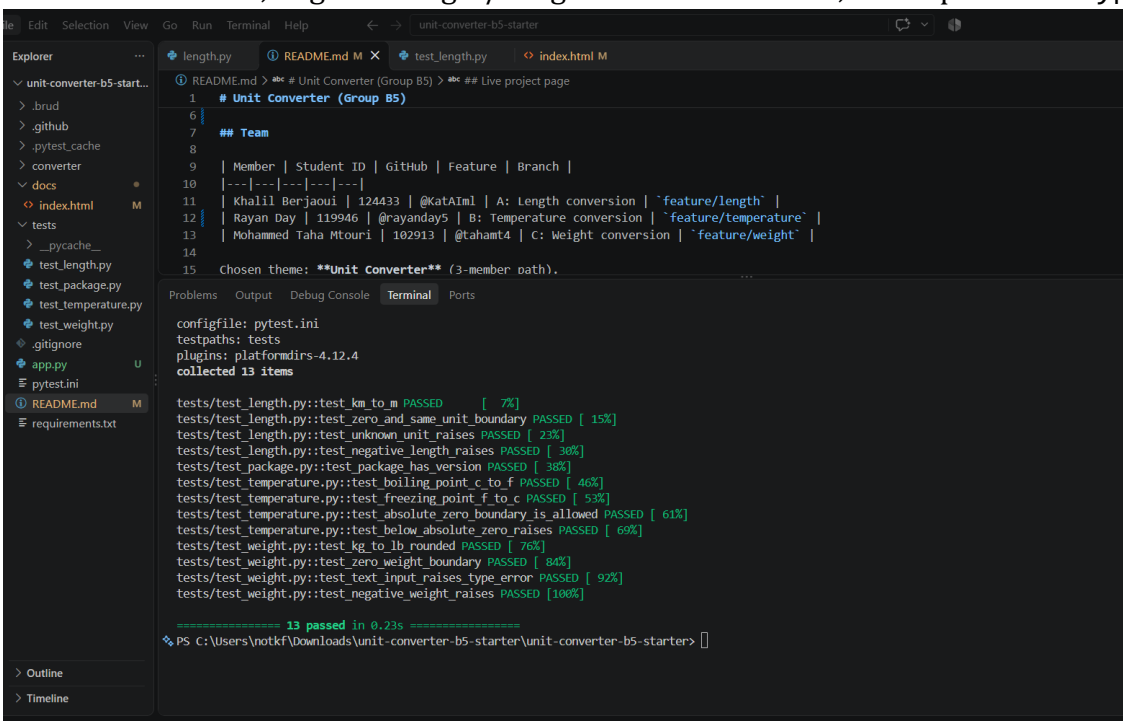


Figure 7. Local run of the complete suite: 13 passed.

6. Continuous Integration

`.github/workflows/tests.yml` (workflow "Tests") runs on every `push` and `pull_request`. Its job `test` runs on `ubuntu-latest` with four steps: check out the code (`actions/checkout`), set up Python 3.12 (`actions/setup-python`), `pip install -r requirements.txt`, and `pytest -v`. **Failed run:** on PR #3, the first run failed with `FAILED tests/test_temperature.py::test_boiling_point_c_to_f - assert 148.0 == 212`. The cause was the Celsius-to-Fahrenheit formula, which subtracted 32 instead of adding it (`value * 9 / 5 - 32`). Rayan inspected the "Run pytest" log, corrected it to `+ 32`, committed "Fix Celsius to Fahrenheit formula" and pushed to the same branch, and the next run passed.

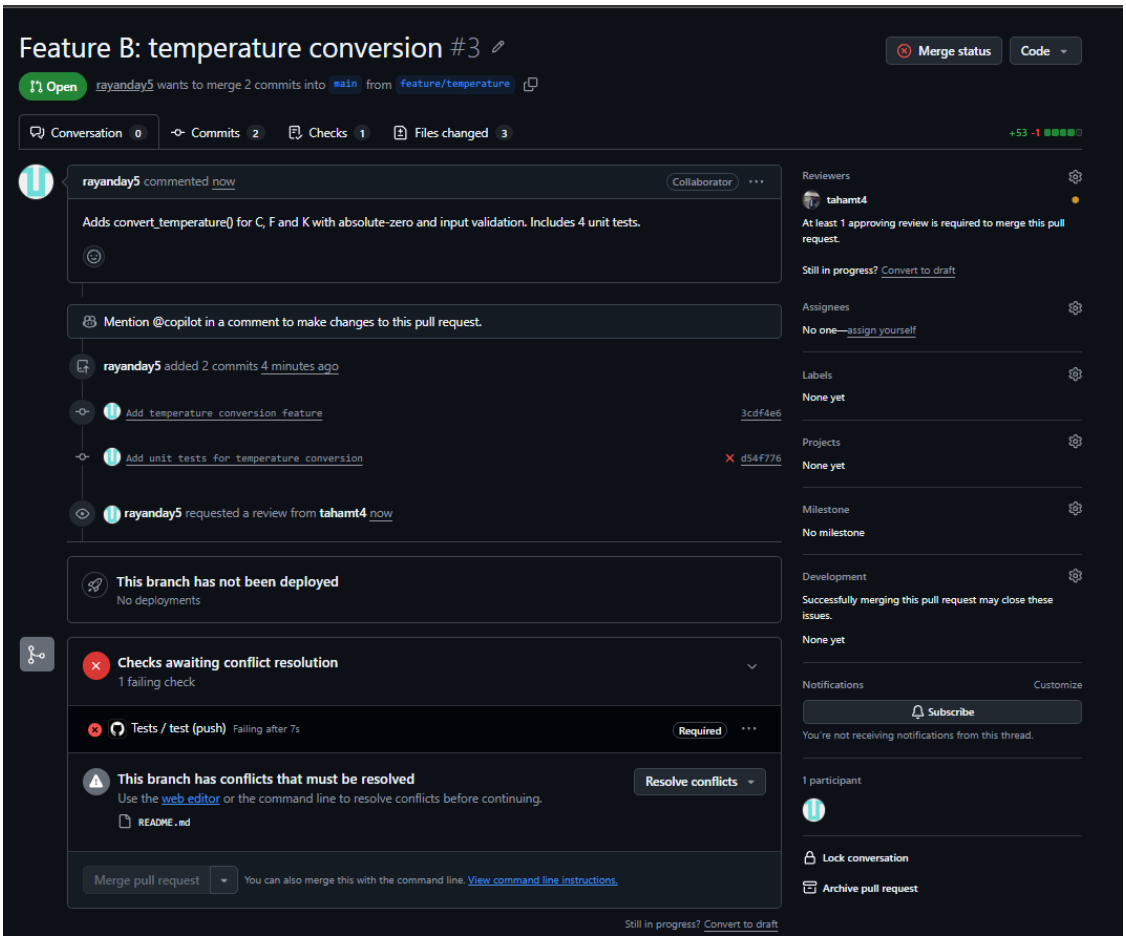


Figure 8. PR #3: required Tests / test check failing, plus the README conflict blocking the merge.

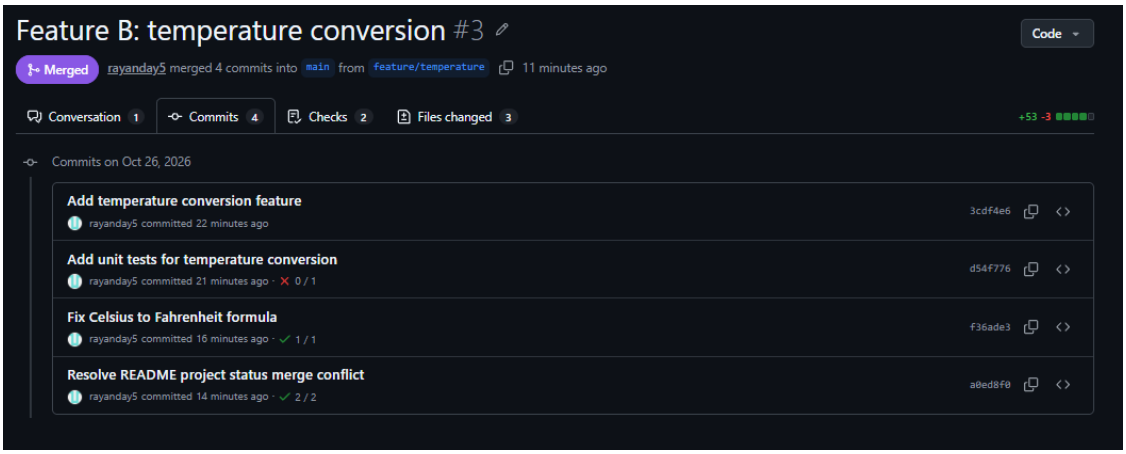


Figure 9. PR #3 commits: the tests commit failed CI (0/1), the formula fix passed (1/1), and the conflict-resolution commit passed (2/2).

7. Ruleset & Protected main

The ruleset `protect-main` is **Active**, targets the default branch (`main`) and has an empty bypass list. It restricts deletions, blocks force pushes and requires a Pull Request with at least **1 approval**. Once the workflow had run successfully, we added **Require status checks to pass** with the `test` check (shown as “Required” on PRs). As a result, every PR showed “Review required” / “Merging is blocked” until another member approved and the `test` check was green; PR #3 was also blocked while its check failed and while the README conflict was unresolved.

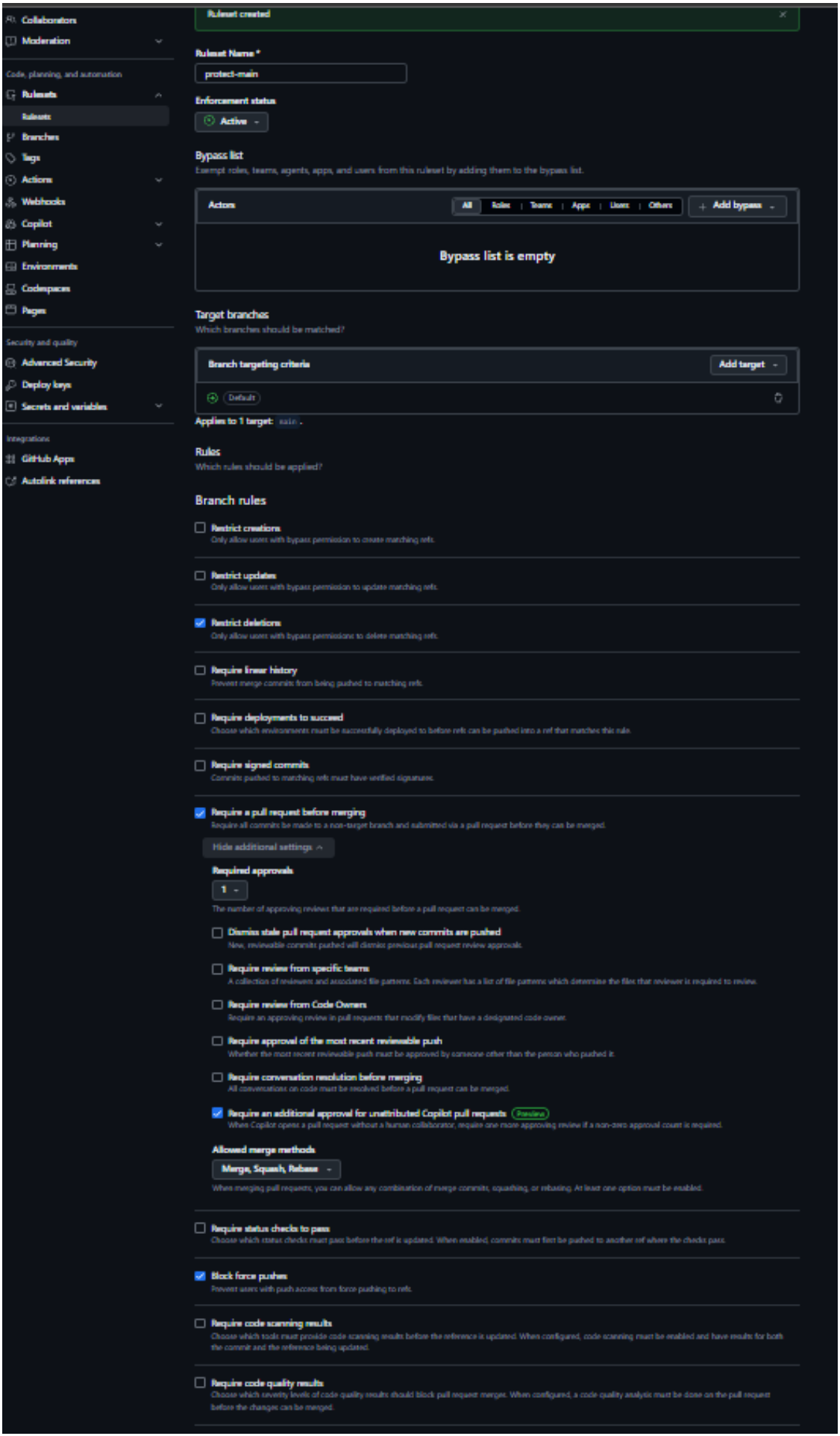

Collaborators
Moderation
Code, planning, and automation
Rulesets
Rulesets
Branches
Tags
Actions
Webhooks
Copilot
Planning
Environments
Codespaces
Pages
Security and quality
Advanced Security
Deploy keys
Secrets and variables
Integrations
GitHub Apps
Autolink references
Ruleset created
Ruleset Name *
protect-main
Enforcement status
Active
Bypass list
Exempt roles, teams, agents, apps, and users from this ruleset by adding them to the bypass list.
Actors
All
Roles
Teams
Apps
Users
Others
Add bypass
Bypass list is empty
Target branches
Which branches should be matched?
Branch targeting criteria
Add target
Default
Applies to 1 target: main.
Rules
Which rules should be applied?
Branch rules
Restrict creations
Only allow users with bypass permission to create matching refs.
Restrict updates
Only allow users with bypass permission to update matching refs.
Restrict deletions
Only allow users with bypass permissions to delete matching refs.
Require linear history
Prevent merge commits from being pushed to matching refs.
Require deployments to succeed
Choose which environments must be successfully deployed to before refs can be pushed into a ref that matches this rule.
Require signed commits
Commits pushed to matching refs must have verified signatures.
Require a pull request before merging
Require all commits be made to a non-target branch and submitted via a pull request before they can be merged.
Hide additional settings
Required approvals
1
The number of approving reviews that are required before a pull request can be merged.
Dismiss stale pull request approvals when new commits are pushed
New, reviewable commits pushed will dismiss previous pull request review approvals.
Require review from specific teams
A collection of reviewers and associated file patterns. Each reviewer has a list of file patterns which determine the files that reviewer is required to review.
Require review from Code Owners
Require an approving review in pull requests that modify files that have a designated code owner.
Require approval of the most recent reviewable push
Whether the most recent reviewable push must be approved by someone other than the person who pushed it.
Require conversation resolution before merging
All conversations on code must be resolved before a pull request can be merged.
Require an additional approval for unattributed Copilot pull requests
Preview
When Copilot opens a pull request without a human collaborator, require one more approving review if a non-zero approval count is required.
Allowed merge methods
Merge, Squash, Rebase
When merging pull requests, you can allow any combination of merge commits, squashing, or rebasing. At least one option must be enabled.
Require status checks to pass
Choose which status checks must pass before the ref is updated. When enabled, commits must first be pushed to another ref where the checks pass.
Block force pushes
Prevent users with push access from force pushing to refs.
Require code scanning results
Choose which tools must provide code scanning results before the reference is updated. When configured, code scanning must be enabled and have results for both the commit and the reference being updated.
Require code quality results
Choose which severity levels of code quality results should block pull request merges. When configured, a code quality analysis must be done on the pull request before the changes can be merged.

Figure 10. Ruleset protect-main: active, default branch, restrict deletions, block force pushes, PR with 1 approval.

8. Continuous Deployment

GitHub Pages is configured as "Deploy from a branch" with `main` and the `/docs` folder, so every merge into `main` triggers the automatic "pages build and deployment" workflow. Our final PR #4 changed the page status from "Version 0.1" to "Version 1.0: all three features merged and tested." and added the live URL to the README. After merging, the Pages workflow ran and the live site showed the new text.

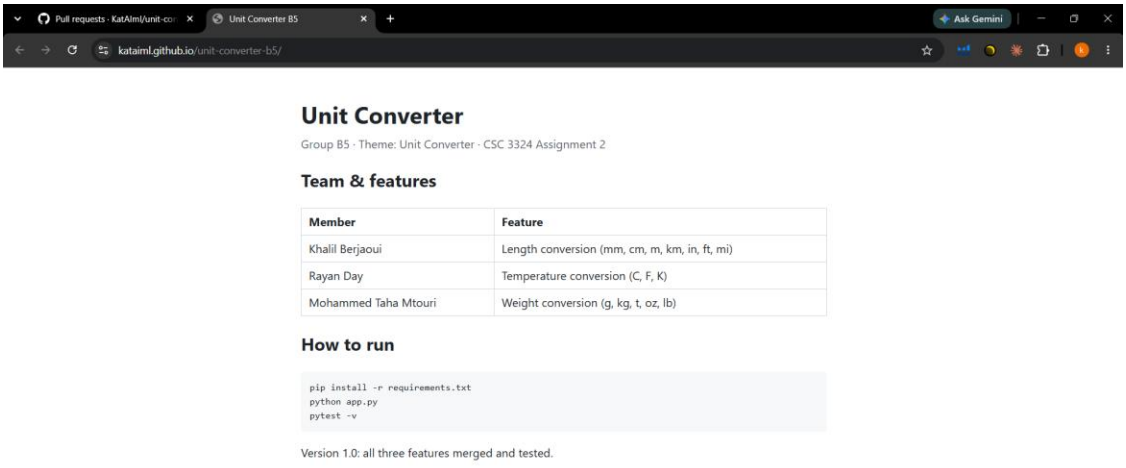


Figure 11. Live GitHub Pages site after merging PR #4, showing Version 1.0.

9. Contribution Summary

Member	Feature	Branch(es)	PRs authored	PRs reviewed	Tests
Khalil	Length, CLI menu, project page	`feature/length`, `feature/final-page`	#1, #4	#2 (Request changes, then approve)	4 length + 1 smoke
Rayan	Temperature	`feature/temperature`	#3	#1, #4	4 temperature
Taha	Weight	`feature/weight`	#2	#3 (conflict-resolution PR)	4 weight

10. Fair Use of AI

We used Claude (an AI assistant) for guidance throughout the assignment: step-by-step Git/GitHub commands and the order of the workflow. The failing CI run was set up on purpose with an incorrect formula so the tests would catch it. Each member ran the commands, commits, Pull Requests and reviews on their own account, we adapted the steps when they did not fit our situation, verified everything by running the tests locally and in GitHub Actions, and edited the report to match what happened in the repository. Each member is responsible for understanding the work under their account.

11. Sources & Reflection

Sources: CSC 3324 Session 2 slides; GitHub Docs (rulesets, pull requests and reviews, GitHub Actions, GitHub Pages); pytest documentation; Claude (see section 10). Our hardest problem was ordering: the required status check can only be selected after the workflow has run once, and the README conflict only appears if one branch is merged before the other merges `origin/main`. We also made a mistake in the first push (the files ended up inside an extra folder, so the workflow and `docs/` were not found) and fixed it before protecting `main`. If a PR, merge, test or workflow behaves unexpectedly, we would first read the failing step in the Actions log, then check that the branch is up to date with `origin/main` and that the files are in the right place, and finally check the ruleset settings.